# 会社管理画面UI設計書

## メタデータ

**title: 会社管理画面UI設計書**
**category: UI設計**
**version: 1.0.0**
**last_updated: 2026-01-23**
**author: Development Team**
**tags: [provider, corporation, list, main]**

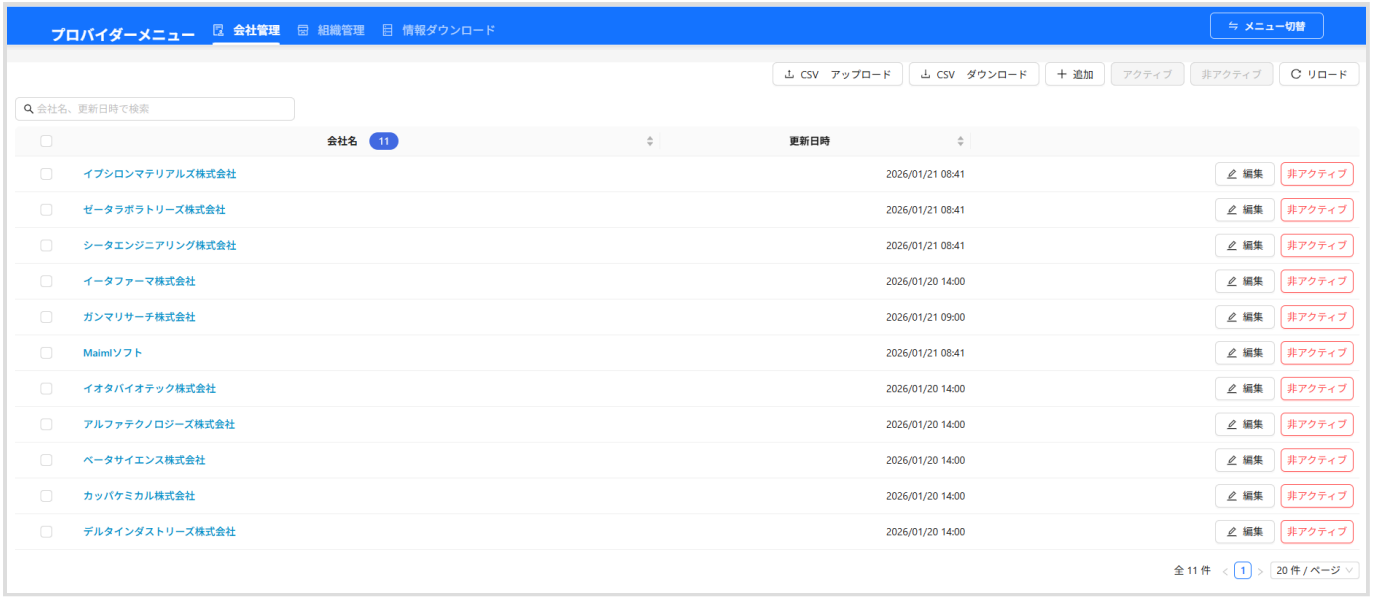
## 関連ドキュメント

- 会社追加ダイアログ
- 会社編集ダイアログ
- マスタ管理API仕様
- 会社テーブルDDL
- 会社管理DB接続計画
- UIポリシー

---

## 2. 画面レイアウト

### 2.1. 画面イメージ




---

# 3. 概要

## 3.1. 目的

プロバイダが会社マスタを管理するためのメイン画面を定義します。会社の一覧表示、検索、フィルタリング、追加、編集、ステータス切替、CSV入出力機能を提供します。

## 3.2. 利用権限

- **Provider（プロバイダ）**: 使用可能

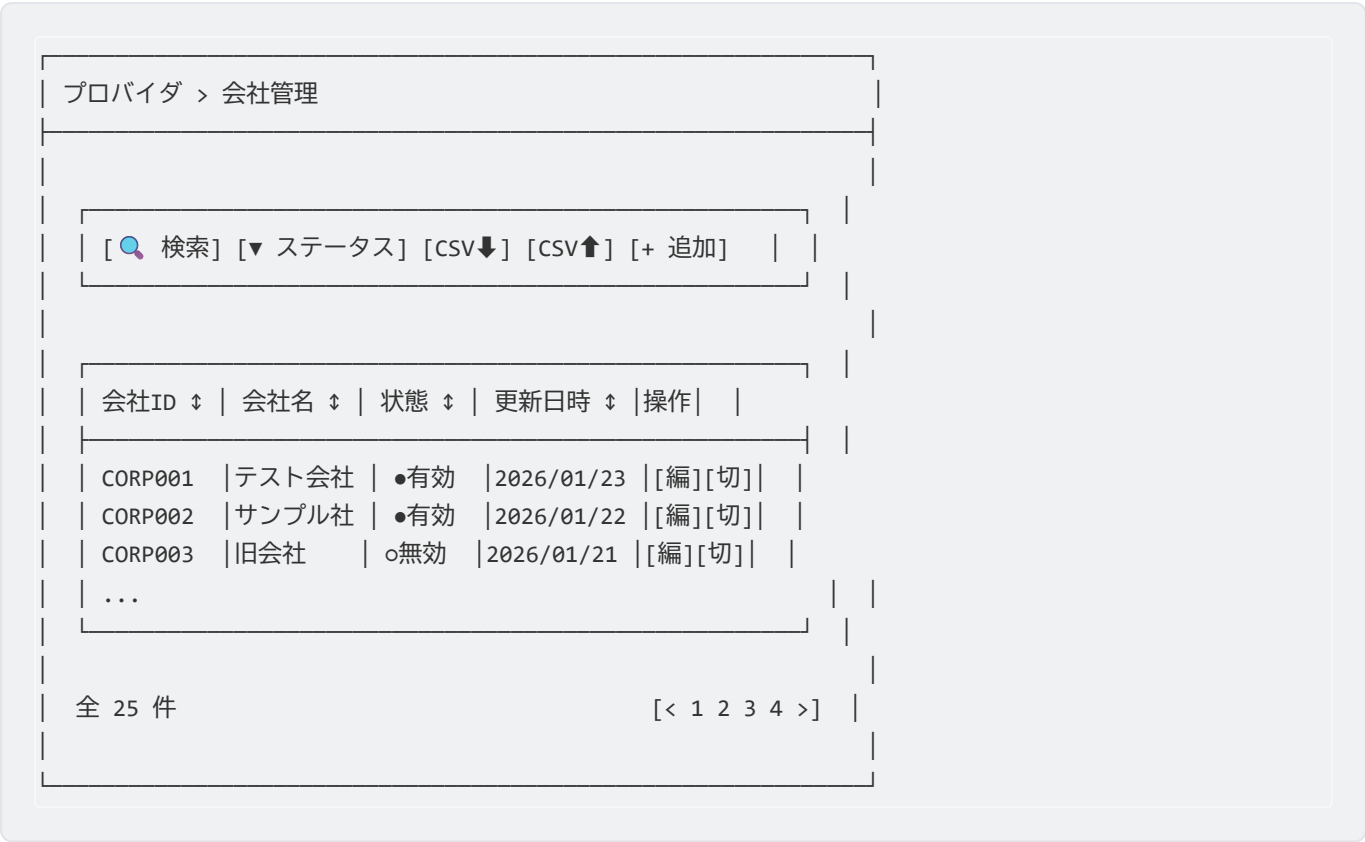
## 3.3. URL

`/provider/corporation`

---

# 2. 画面構成

## 2.1. レイアウト




---

# 3. ツールバー

## 3.1. 配置

**左側**: 検索・フィルタUI **右側**: CSV操作、追加ボタン（右から主要な順）

## 3.2. コンポーネント

### 3.2.1. 検索入力

**コンポーネント**: `<Input>`

**仕様**:

| 項目 | 値 |
|---|---|
| プレースホルダー | 「会社ID、会社名で検索」 |
| 前置アイコン | `<SearchOutlined />` |
| 幅 | 250px |

| | |
|---|---|
| クリアボタン | 有効（allowClear） |

**検索対象**:

- 会社ID（部分一致）
- 会社名（部分一致）

### 3.2.2. ステータスフィルター

**コンポーネント**: `<Select>`

**仕様**:

| 項目 | 値 |
|---|---|
| プレースホルダー | 「ステータス」 |
| 幅 | 150px |
| クリアボタン | 有効（allowClear） |

**選択肢**:

- すべて（フィルタなし）
- アクティブ
- 非アクティブ

### 3.2.3. CSVダウンロードボタン

**コンポーネント**: `<Button>`

**仕様**:

| 項目 | 値 |
|---|---|
| ラベル | 「CSV」 |
| タイプ | `default` |
| アイコン | `<DownloadOutlined />` |
| サイズ | 省略（デフォルト） |

**動作**:

1. 現在のフィルタ・検索条件を適用

2. API呼び出し: `GET /api/v1/provider/corporation/download-csv`
3. CSVファイルをダウンロード
4. ファイル名: `corporation_YYYYMMDD_HHmmss.csv`

### 3.2.4. CSVアップロードボタン

**コンポーネント**: `<Upload>` + `<Button>`

**仕様**:

| 項目 | 値 |
|---|---|
| ラベル | 「CSV一括登録」 |
| タイプ | `default` |
| アイコン | `<UploadOutlined />` |
| サイズ | 省略（デフォルト） |
| 許可形式 | .csv |

**動作**:

1. CSVファイル選択
2. アップロード前にプレビュー表示（オプション）
3. API呼び出し: `POST /api/v1/provider/corporation/upload-csv`
4. 結果表示（成功件数、失敗件数、エラー詳細）

### 3.2.5. 追加ボタン

**コンポーネント**: `<Button>`

**仕様**:

| 項目 | 値 |
|---|---|
| ラベル | 「追加」 |
| タイプ | `primary` |
| アイコン | `<PlusOutlined />` |
| サイズ | 省略（デフォルト） |

**動作**:

1. 会社追加ダイアログを開く
2. ダイアログで会社情報を入力
3. 保存後、一覧を再取得

---

# 4. データテーブル

## 4.1. テーブル仕様

**コンポーネント**: `<Table>`

**主要設定**:

| 項目 | 値 |
|---|---|
| rowKey | `corporationId` |
| pagination | 有効 |
| pageSize | 20（デフォルト） |
| pageSizeOptions | [10, 20, 50, 100] |
| scroll | `{ x: 1000 }` |

## 4.2. カラム定義

### 4.2.1. 会社ID

**プロパティ**:

| 項目 | 値 |
|---|---|
| title | 「会社ID」 |
| dataIndex | `corporationId` |
| key | `corporationId` |
| width | 150px |
| align | `left` |

| | |
|---|---|
| sorter | 有効 |

### 4.2.2. 会社名

**プロパティ**:

| 項目 | 値 |
|---|---|
| title | 「会社名」 |
| dataIndex | `corporationName` |
| key | `corporationName` |
| width | 250px |
| align | `left` |
| sorter | 有効 |

### 4.2.3. 状態

**プロパティ**:

| 項目 | 値 |
|---|---|
| title | 「状態」 |
| dataIndex | `isActive` |
| key | `isActive` |
| width | 120px |
| align | `center` |
| sorter | 有効 |

**レンダリング**:

```
render: (isActive: boolean) => (
  <Badge
    status={isActive ? 'success' : 'default'}
    text={isActive ? 'アクティブ' : '非アクティブ'}
  />
)
```

### 4.2.4. 更新日時

**プロパティ**:

| 項目 | 値 |
| --- | --- |
| title | 「更新日時」 |
| dataIndex | `updateAt` |
| key | `updateAt` |
| width | 180px |
| align | `right` |
| sorter | 有効 |
| defaultSortOrder | `descend` |

**レンダリング**:

```
render: (date: string) => {
  return date ? new Date(date).toLocaleString('ja-JP') : '-';
}
```

### 4.2.5. 操作

**プロパティ**:

| 項目 | 値 |
| --- | --- |
| title | 「操作」 |
| key | `action` |
| width | 200px |
| align | `center` |
| fixed | `right` |

**レンダリング**:

```
render: (_, record) => (
  <Space size="small">
```

```
    <Button
      type="default"
      size="small"
      icon={<EditOutlined />}
      onClick={() => handleEdit(record.corporationId)}
    >
      編集
    </Button>
    <Button
      type="default"
      size="small"
      style={record.isActive ? {
        color: token.colorError,
        borderColor: token.colorError
      } : {
        color: token.colorSuccess,
        borderColor: token.colorSuccess
      }}
      onClick={() => handleStatusToggle(record.corporationId, !record.isActive)}
    >
      {record.isActive ? '非アクティブ化' : 'アクティブ化'}
    </Button>
  </Space>
)
```

---

# 5. ページネーション

## 5.1. 仕様

**コンポーネント**: `<Table>` の pagination プロパティ

**設定**:

```
pagination={{
  pageSize: 20,
  showSizeChanger: true,
  showTotal: (total) => `全 ${total} 件`,
  pageSizeOptions: ['10', '20', '50', '100']
}}
```

---

# 6. APIリクエスト仕様

## 6.1. 一覧取得

**エンドポイント**: `GET /api/v1/provider/corporation`

**クエリパラメータ**:

| パラメータ | 型 | 説明 |
|---|---|---|
| search | string | 検索テキスト（会社ID、会社名） |
| is_active | boolean | アクティブフィルタ |
| sort_by | string | ソートフィールド（corporation_id, corporation_name, update_at） |
| sort_order | string | ソート順（asc, desc） |
| page | number | ページ番号（デフォルト: 1） |
| limit | number | 1ページあたりの件数（デフォルト: 20） |

**リクエスト例**:

```
GET /api/v1/provider/corporation?search=テスト
&is_active=true&sort_by=update_at&sort_order=desc&page=1&limit=20
Authorization: Bearer {jwt_token}
```

**レスポンス**:

```
{
  "success": true,
  "data": [
    {
      "corporationId": "CORP001",
      "corporationName": "テスト会社",
      "isActive": true,
      "createAt": "2026-01-20T10:30:00.000Z",
      "updateAt": "2026-01-23T10:30:00.000Z"
    }
  ],
  "pagination": {
    "page": 1,
    "limit": 20,
```

```
    "totalItems": 25,
    "totalPages": 2,
    "hasNext": true,
    "hasPrev": false
  },
  "meta": {
    "timestamp": "2026-01-23T10:30:00.000Z",
    "requestId": "req_abc123"
  }
}
```

## 6.2. ステータス更新

**エンドポイント**: `PATCH /api/v1/provider/corporation/:corporationId/status`

**リクエストボディ**:

```
{
  "isActive": false
}
```

**レスポンス**:

```
{
  "success": true,
  "data": {
    "corporationId": "CORP001",
    "corporationName": "テスト会社",
    "isActive": false,
    "createAt": "2026-01-20T10:30:00.000Z",
    "updateAt": "2026-01-23T11:00:00.000Z"
  },
  "meta": {
    "timestamp": "2026-01-23T11:00:00.000Z",
    "requestId": "req_def456"
  }
}
```

## 6.3. CSVダウンロード

**エンドポイント**: `GET /api/v1/provider/corporation/download-csv`

**クエリパラメータ**: 一覧取得と同じ（search, is_active）

**レスポンス**: CSVファイル（UTF-8 BOM付き）

**CSV形式**:

```
会社ID,会社名,状態
CORP001,テスト会社,アクティブ
CORP002,サンプル社,アクティブ
CORP003,旧会社,非アクティブ
```

## 6.4. CSVアップロード

**エンドポイント**: `POST /api/v1/provider/corporation/upload-csv`

**リクエストヘッダー**:

```
Content-Type: multipart/form-data
Authorization: Bearer {jwt_token}
```

**レスポンス**:

```
{
  "success": true,
  "data": {
    "successCount": 18,
    "failureCount": 2,
    "errors": [
      {
        "row": 5,
        "corporationId": "CORP999",
        "error": "会社IDが重複しています"
      },
      {
        "row": 12,
        "corporationId": "",
        "error": "会社IDは必須です"
      }
    ]
```

```
  },
  "meta": {
    "timestamp": "2026-01-23T11:00:00.000Z",
    "requestId": "req_ghi789"
  }
}
```

---

# 7. イベントハンドリング

## 7.1. 検索

**トリガー**: 検索入力の変更（デバウンス500ms）

**処理**:

1. 検索テキストを状態に保存
2. 500msのデバウンス
3. API呼び出し（page=1にリセット）

## 7.2. ステータスフィルター

**トリガー**: ステータス選択の変更

**処理**:

1. 選択値を状態に保存
2. API呼び出し（page=1にリセット）

## 7.3. ソート

**トリガー**: カラムヘッダークリック

**処理**:

1. ソートフィールドとソート順を状態に保存
2. API呼び出し（page維持）

### 7.4. ページ変更

**トリガー**: ページネーションクリック

**処理**:

1. ページ番号を状態に保存
2. API呼び出し

### 7.5. 編集ボタンクリック

**処理**:

1. 会社IDを保存
2. 編集ダイアログを開く
3. 保存後、一覧を再取得

### 7.6. ステータス切替ボタンクリック

**処理**:

1. 確認ダイアログ表示（非アクティブ化の場合）
2. API呼び出し: `PATCH /api/v1/provider/corporation/:corporationId/status`
3. 成功時、一覧を再取得
4. 失敗時、エラーメッセージ表示

---

## 8. 実装例（TypeScript + Next.js + Ant Design）

```
// frontend/src/app/provider/corporation/page.tsx
'use client';

import React, { useState, useEffect } from 'react';
import {
  Table,
  Input,
```

```
  Select,
  Button,
  Space,
  Badge,
  message,
  Modal,
  Upload,
  theme
} from 'antd';
import {
  SearchOutlined,
  PlusOutlined,
  EditOutlined,
  DownloadOutlined,
  UploadOutlined
} from '@ant-design/icons';
import type { ColumnsType, TablePaginationConfig } from 'antd/es/table';
import apiClient from '@/utils/apiClient';
import CorporationAddDialog from '@/components/Provider/CorporationAddDialog';
import CorporationEditDialog from '@/components/Provider/CorporationEditDialog';

const { useToken } = theme;

interface Corporation {
  corporationId: string;
  corporationName: string;
  isActive: boolean;
  createAt: string;
  updateAt: string;
}

export default function CorporationManagementPage() {
  const { token } = useToken();
  const [data, setData] = useState<Corporation[]>([]);
  const [loading, setLoading] = useState(false);
  const [searchText, setSearchText] = useState('');
  const [statusFilter, setStatusFilter] = useState<boolean | undefined>(undefined);
  const [sortField, setSortField] = useState('update_at');
  const [sortOrder, setSortOrder] = useState<'asc' | 'desc'>('desc');
  const [pagination, setPagination] = useState({
    current: 1,
    pageSize: 20,
    total: 0
  });

  const [isAddDialogOpen, setIsAddDialogOpen] = useState(false);
  const [isEditDialogOpen, setIsEditDialogOpen] = useState(false);
  const [editingCorporationId, setEditingCorporationId] = useState<string | null>(null);

  // データ取得
```

```
  const fetchData = async () => {
    setLoading(true);
    try {
      const response = await apiClient.get('/api/v1/provider/corporation', {
        params: {
          search: searchText || undefined,
          is_active: statusFilter,
          sort_by: sortField,
          sort_order: sortOrder,
          page: pagination.current,
          limit: pagination.pageSize
        }
      });

      if (response.data.success) {
        setData(response.data.data);
        setPagination({
          ...pagination,
          total: response.data.pagination.totalItems
        });
      }
    } catch (error: any) {
      message.error('データの取得に失敗しました');
    } finally {
      setLoading(false);
    }
  };

  useEffect(() => {
    fetchData();
  }, [searchText, statusFilter, sortField, sortOrder, pagination.current,
pagination.pageSize]);

  // 検索（デバウンス）
  useEffect(() => {
    const timer = setTimeout(() => {
      setPagination({ ...pagination, current: 1 });
    }, 500);
    return () => clearTimeout(timer);
  }, [searchText]);

  // ステータス切替
  const handleStatusToggle = (corporationId: string, newStatus: boolean) => {
    const action = newStatus ? 'アクティブ化' : '非アクティブ化';
    Modal.confirm({
      title: `会社を${action}しますか？`,
      content: `${corporationId} を${action}します。`,
      okText: 'はい',
      cancelText: 'いいえ',
      onOk: async () => {
```

```
        try {
          await apiClient.patch(
            `/api/v1/provider/corporation/${corporationId}/status`,
            { isActive: newStatus }
          );
          message.success(`会社を${action}しました`);
          fetchData();
        } catch (error: any) {
          message.error(`${action}に失敗しました`);
        }
      }
    });
  };

  // CSVダウンロード
  const handleDownloadCsv = async () => {
    try {
      const response = await apiClient.get(
        '/api/v1/provider/corporation/download-csv',
        {
          params: {
            search: searchText || undefined,
            is_active: statusFilter
          },
          responseType: 'blob'
        }
      );

      const blob = new Blob([response.data], { type: 'text/csv' });
      const url = window.URL.createObjectURL(blob);
      const link = document.createElement('a');
      link.href = url;
      link.download = `corporation_${new Date().toISOString().replace(/[:.]/g, '-')}.csv`;
      document.body.appendChild(link);
      link.click();
      document.body.removeChild(link);
      window.URL.revokeObjectURL(url);

      message.success('CSVをダウンロードしました');
    } catch (error) {
      message.error('CSVダウンロードに失敗しました');
    }
  };

  // CSVアップロード
  const handleUploadCsv = async (file: File) => {
    const formData = new FormData();
    formData.append('file', file);

    try {
```

```
      const response = await apiClient.post(
        '/api/v1/provider/corporation/upload-csv',
        formData,
        {
          headers: {
            'Content-Type': 'multipart/form-data'
          }
        }
      );

      if (response.data.success) {
        const { successCount, failureCount, errors } = response.data.data;

        if (failureCount === 0) {
          message.success(`${successCount}件のデータを登録しました`);
        } else {
          Modal.warning({
            title: 'CSV登録結果',
            content: (
              <div>
                <p>成功: {successCount}件</p>
                <p>失敗: {failureCount}件</p>
                {errors.length > 0 && (
                  <div>
                    <p>エラー詳細:</p>
                    <ul>
                      {errors.map((err: any, idx: number) => (
                        <li key={idx}>
                          行{err.row}: {err.error}
                        </li>
                      ))}
                    </ul>
                  </div>
                )}
              </div>
            )
          });
        }
        fetchData();
      }
    } catch (error: any) {
      message.error('CSVアップロードに失敗しました');
    }

    return false; // prevent default upload behavior
  };

  // テーブルカラム定義
  const columns: ColumnsType<Corporation> = [
    {
```

```
    title: '会社ID',
    dataIndex: 'corporationId',
    key: 'corporationId',
    width: 150,
    align: 'left',
    sorter: true
  },
  {
    title: '会社名',
    dataIndex: 'corporationName',
    key: 'corporationName',
    width: 250,
    align: 'left',
    sorter: true
  },
  {
    title: '状態',
    dataIndex: 'isActive',
    key: 'isActive',
    width: 120,
    align: 'center',
    sorter: true,
    render: (isActive: boolean) => (
      <Badge
        status={isActive ? 'success' : 'default'}
        text={isActive ? 'アクティブ' : '非アクティブ'}
      />
    )
  },
  {
    title: '更新日時',
    dataIndex: 'updateAt',
    key: 'updateAt',
    width: 180,
    align: 'right',
    sorter: true,
    defaultSortOrder: 'descend',
    render: (date: string) => (date ? new Date(date).toLocaleString('ja-JP') : '-')
  },
  {
    title: '操作',
    key: 'action',
    width: 200,
    align: 'center',
    fixed: 'right',
    render: (_, record) => (
      <Space size="small">
        <Button
          type="default"
          size="small"
```

```
            icon={<EditOutlined />}
            onClick={() => {
              setEditingCorporationId(record.corporationId);
              setIsEditDialogOpen(true);
            }}
          >
            編集
          </Button>
          <Button
            type="default"
            size="small"
            style={
              record.isActive
                ? {
                    color: token.colorError,
                    borderColor: token.colorError
                  }
                : {
                    color: token.colorSuccess,
                    borderColor: token.colorSuccess
                  }
            }
            onClick={() => handleStatusToggle(record.corporationId, !record.isActive)}
          >
            {record.isActive ? '非アクティブ化' : 'アクティブ化'}
          </Button>
        </Space>
      )
    }
  ];

  // テーブル変更ハンドラ
  const handleTableChange = (
    newPagination: TablePaginationConfig,
    filters: any,
    sorter: any
  ) => {
    if (sorter.field) {
      setSortField(sorter.field);
      setSortOrder(sorter.order === 'ascend' ? 'asc' : 'desc');
    }
    setPagination({
      current: newPagination.current || 1,
      pageSize: newPagination.pageSize || 20,
      total: pagination.total
    });
  };

  return (
    <div style={{ padding: 24 }}>
```

```
      <h2>会社管理</h2>

      {/* ツールバー */}
      <Space style={{ marginBottom: 16, width: '100%', justifyContent: 'space-between' }}>
        <Space>
          <Input
            placeholder="会社ID、会社名で検索"
            prefix={<SearchOutlined />}
            style={{ width: 250 }}
            value={searchText}
            onChange={(e) => setSearchText(e.target.value)}
            allowClear
          />
          <Select
            placeholder="ステータス"
            style={{ width: 150 }}
            value={statusFilter}
            onChange={setStatusFilter}
            allowClear
            options={[
              { label: 'アクティブ', value: true },
              { label: '非アクティブ', value: false }
            ]}
          />
        </Space>

        <Space>
          <Button
            icon={<DownloadOutlined />}
            onClick={handleDownloadCsv}
          >
            CSV
          </Button>
          <Upload
            beforeUpload={handleUploadCsv}
            showUploadList={false}
            accept=".csv"
          >
            <Button icon={<UploadOutlined />}>CSV一括登録</Button>
          </Upload>
          <Button
            type="primary"
            icon={<PlusOutlined />}
            onClick={() => setIsAddDialogOpen(true)}
          >
            追加
          </Button>
        </Space>
      </Space>
```

```
      {/* テーブル */}
      <Table
        columns={columns}
        dataSource={data}
        rowKey="corporationId"
        loading={loading}
        pagination={{
          current: pagination.current,
          pageSize: pagination.pageSize,
          total: pagination.total,
          showSizeChanger: true,
          showTotal: (total) => `全 ${total} 件`,
          pageSizeOptions: ['10', '20', '50', '100']
        }}
        scroll={{ x: 1000 }}
        onChange={handleTableChange}
      />

      {/* ダイアログ
```